From screenplays to ranked promos.

KUKU
PROMO FINDER

A notebook that helps an editor find 20–60-second clips with an immediate hook, understandable stakes and an unresolved ending. It selects footage and cut points; the editor makes the final decision.

50
episodes · 3,727 source entries parsed

51
candidates passed the technical checks

35
groups · 16 alternative cuts retained

01 Parser → 02 Candidate generation → 03 Deduplication → 04 Ranking

01 Parser

Python reads scenes, dialogue, actions, camera cues and sound. Each entry keeps its original text and episode-relative timestamps, and receives a stable source ID.

Overlapping audio and visuals stay separate. Missing or malformed fields raise an error; source timing inconsistencies are flagged rather than silently corrected.

Output: structured episodes and ID-labelled screenplay text. This step uses rules, not an LLM.

02 Candidate generation

Gemini 3.7 reads 10 episodes at a time, advancing by 5. The full pack needs 9 overlapping windows. Each response may propose up to 8 promos; there is no per-episode quota.

The model chooses source IDs for each segment. Code resolves exact timestamps and checks the total duration, episode boundaries and repeated footage.

Selection: a hook within 3 seconds, enough context for a new viewer, and a specific unanswered ending.

03 Deduplication

Overlapping windows can find the same moment. A rule compares footage: corresponding segments must use the same episodes and order, sharing at least 80% of the shorter segment.

Titles and dialogue similarity are not used. The first occurrence represents a group, while every alternative cut remains available for judging.

Result: 51 candidates become 35 groups. The same rules apply to another series in this input format.

04 Ranking

A separate judge sees the selected screenplay excerpts, without the generator’s titles, explanations or story summaries. It scores each cut from 0–4 and cites source evidence.

Code weights **hook 30%**, **cliffhanger 30%**, **clarity 25%** and **escalation 15%**. The best cut wins each group; a final comparison orders the leading shortlist.

Quality gate: a score below 2 on hook, cliffhanger or clarity flags the promo for review.

Completed run. 35 groups ranked; 27 passed the configured core-quality threshold. The judge chose a stronger alternative cut in 2 groups. These are editorial judgments, not measured advertising results.

Scope: all episodes form one search pool. A promo may be continuous or combine ordered segments across episodes. The tool finds clips and cut points; it does not create edits, music or overlays. All ranking weights and thresholds are editable assumptions.

One clip, traced through the pipeline.

An independently written, illustrative example. Its score and rank below are not results from the assignment run.

EXAMPLE · EPISODE 1 **00:00.00 → 00:54.00 · 54 seconds**

An engineer discovers sabotaged flood controls. A colleague locks the control-room door as the water rises.

```
[E001-B0001 | 00:00–00:03] ACTION: Emergency stop fails.
… intermediate dialogue, actions and camera cues preserved …
[E001-B0012 | 00:49–00:54] JON: “Put down the radio.”
```

01 → 02 · Parse and select

The parser links each ID to its timestamps. Selecting B0001 through B0012 includes **all footage from 00:00 to 00:54**, not just those two entries.

03 · Check overlap

A second cut, 00:01–00:54, shares 53 of its 53 seconds with the first: **100% overlap of the shorter cut**. Both versions enter the same group for judging.

04 · Editor-facing record

Hook: the emergency stop fails in the first three seconds. **Cliffhanger:** can Mara get past Jon and stop the flood? **Why it works:** a failed safety system reveals sabotage, then the locked room leaves the engineer facing an unresolved threat.

Illustrative posting rationale: immediate danger, clear stakes and a concrete unresolved problem.

ILLUSTRATIVE RANK #1

88.75 / 100

Hook 4 · Cliffhanger 3 · Clarity 4 · Escalation 3
100 × (4×0.30 + 3×0.30 + 4×0.25 + 3×0.15) ÷ 4
Real judgments retain criterion evidence and concerns.

What is checked—and what can fail

Code checks facts. IDs, durations, boundaries, overlap and judge citations are validated. Automated tests mock API calls; passing them does not prove a promo is compelling.

Real spot-check. The first judge batch scored identical cuts consistently and noticed a 0.1-second fragment of preceding dialogue. Static staredowns scored below an unresolved proposal.

Human review matters. One clip received 100/100 despite subjective concerns. The judge sees text, not finished video, and may share the generator’s biases. Word timing and differently assembled montages remain limitations.

How it runs on the next show

Change the input path. Models, episode window size, stride, candidate count, overlap threshold and ranking weights are notebook parameters. The screenplay format stays the same.

Resume instead of restarting. Successful calls are cached. This run used 9 generation windows, 7 scoring batches and 1 shortlist comparison. Retries add attempts; responses retain token usage.

Free-tier budgets. A saved ledger tracks attempts across the supplied keys, which use separate projects. It cannot see external requests. At larger volume, budget tokens per show, queue unfinished work and keep an editor in the loop.

AI use & one correction. Codex assisted planning, code, debugging and documentation; Gemini generated and judged candidates. An early 12,000-token limit left too little room after thinking and produced truncated JSON. Finish-status checks now reject incomplete responses. The completed run uses medium thinking and a 65,536-token maximum budget.

Reproduce: configure keys in **.env**, set the input path and run **solution.ipynb**. Final exports contain every segment’s episode and timestamps, hook, cliffhanger, explanation and posting rationale. Confidential inputs and outputs stay outside the public repo. Scores are editorial judgments; advertising performance requires audience testing.